

Labortory 2

Spatial Filtering and Thresholding

ECE 7410: Image Processing

July 2nd 2025

Group Member	Student ID
Keegan Rolls	202004149
Nicholas Philpott	202018248

Spatial Filtering

Smoothing

1. Download the test images (*img1.png* and *img2.png*) from Brightspace under Lab 2.



2. Read *img1.png*, convert it to a grayscale image, and display.



See the following image

3. Develop and display a function that will perform spatial filtering (i.e., convolution) on an $M \times N$ pixel image using a general $m \times n$ kernel. The function should take an image and a kernel as inputs and return the filtered image. Do **NOT** use internal functions for this (e.g., MATLAB's **imfilter** function). (**hint:** zero padding the edges of the image is required).
4. Filter *img1.png* with each of the five filters below (Kernel 1 to Kernel 5) using your function and display the outputs:
 1. $\text{Kernel_1} = \left(\frac{1}{9}\right) * \text{ones}(0)$
See Figure 1 below.
 2. $\text{Kernel_2} = \left(\frac{1}{49}\right) * \text{ones}(7)$
See Figure 2 below.
 3. $\text{Kernel_3} = \text{fspecial}(\text{'average'}, [7, 7])$
See Figure 3 below.
 4. $\text{Kernel_4} = \text{fspecial}(\text{'gaussian'}, [3, 3], 0.5)$
See Figure 4 below.
 5. $\text{Kernel_5} = \text{fspecial}(\text{'gaussian'}, [7, 7], 1.2)$
See Figure 5 below.
5. Develop and display a function that performs median filtering. A median filter will replace the pixel intensity with the median intensity of the pixels overlapped by the kernel. The function should take the image and the size of the kernel as inputs. (**hint:** zero padding the edges of the image is required).
6. Apply the following two median filters to *img1.png* and display the outputs:



Figure 1: `img1.png` with `Kernel_1` applied



Figure 2: `img1.png` with `Kernel_2` applied



Figure 3: `img1.png` with `Kernel_3` applied



Figure 4: `img1.png` with `Kernel_4` applied



Figure 5: `img1.png` with `Kernel_5` applied

1. Median filter with kernel size 3×3
See **Figure 6** below.
2. Median filter with kernel size 5×5
See **Figure 7** below.
7. Compare and discuss the smoothing effects obtained by `Kernel_1` to `Kernel_5` and the two median filters.
...
8. Discuss the effect the smoothing kernel size has on the output images in your experiments.
...

Sharpening

1. Use the function you developed in **Smoothing: Step 3** and apply the following three sharpening filters (`Kernel_6` to `Kernel_8`) to `img2.png`.

1. $\text{Kernel_6} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & -2 \\ -1 & 0 & 1 \end{bmatrix}$

See **Figure 8** below.

2. $\text{Kernel_7} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

See **Figure 9** below.

3. `Kernel_8 = fspecial('log',3)`



Figure 6: `img1.png` with median filter 3×3 applied



Figure 7: `img1.png` with median filter 5×5 applied

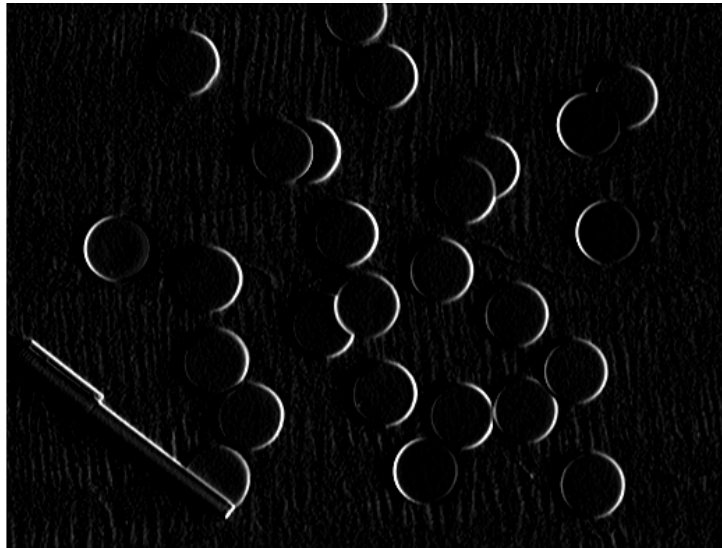


Figure 8: `img2.png` with `Kernel_6` applied

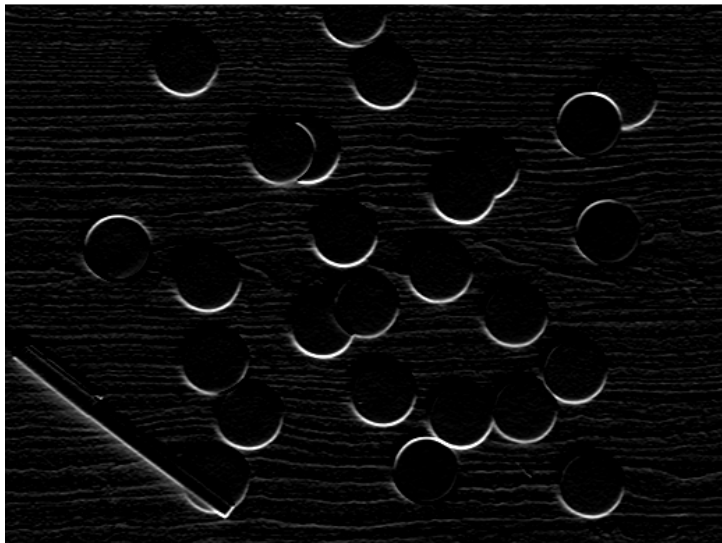


Figure 9: `img2.png` with `Kernel_7` applied

See Figure 10 below.

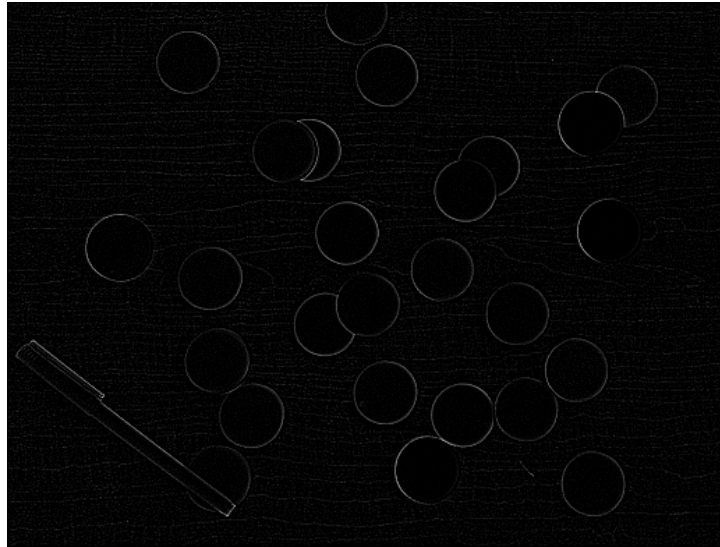


Figure 10: `img2.png` with `Kernel_8` applied

2. Compare and discuss the outputs obtained by *Kernel_6* to *Kernel_8*

...

Thresholding

The following equations are from the lab manual.

$$\omega_0(t) = \sum_{i=0}^{t-1} p(i) \quad \mu_0(t) = \sum_{i=0}^{t-1} ip(i)/\omega_0 \quad \omega_1(t) = \sum_{i=t}^{L-1} p(i) \quad \mu_1(t) = \sum_{i=t}^{L-1} ip(i)/\omega_1 \quad (1)$$

$$\sigma_b^2(t) = \omega_1\omega_2 [\mu_0 - \mu_1]^2 \quad (2)$$

1. Download the test images (*img3.png* and *img4.png*) from Brightspace under Lab 2 and save them in your working directory.



2. Develop and display code to calculate the class probabilities and class means, as well as the inter-class variance (equations 1 and 2) as a complete function to calculate the optimal threshold value for any given image.

%% Q2: Function to calculate optimal threshold using Otsu's method

```
function optimal_threshold = calculate_optimal_threshold(img)
```

```
    % Convert to grayscale if needed
```

```
    if size(img, 3) == 3
```

```
        img = rgb2gray(img);
```

```
    end
```

```
    % Convert to double for calculations
```

```
    img = double(img);
```

```
    % Calculate histogram
```

```
    [counts, ~] = hist(img(:), 0:255);
```

```
    % Calculate probabilities
```

```
    p = counts / sum(counts);
```

```
    % Initialize variables
```

```
    L = 256; % Number of gray levels (0-255)
```

```
    max_variance = 0;
```

```
    optimal_threshold = 0;
```

```
    % Try all possible thresholds
```

```
    for t = 1:(L - 1)
```

```
        % Calculate class probabilities
```

```
        omega_0 = sum(p(1:t));
```

```
        omega_1 = sum(p(t + 1:L));
```

```
        % Skip if one class is empty
```

```
        if omega_0 == 0 || omega_1 == 0
```

```

        continue;
    end

    % Calculate class means
    mu_0 = sum((0:(t - 1)) .* p(1:t)) / omega_0;
    mu_1 = sum((t:(L - 1)) .* p(t + 1:L)) / omega_1;

    % Calculate inter-class variance
    sigma_b_squared = omega_0 * omega_1 * (mu_0 - mu_1) ^ 2;

    % Update optimal threshold if this variance is larger
    if sigma_b_squared > max_variance
        max_variance = sigma_b_squared;
        optimal_threshold = t - 1; % Convert to 0-255 range
    end
end
end
end

```

3. Calculate and report the optimal threshold values for *img3.png* and *img4.png* found using your code in **Thresholding: Step 2**.

Upon executing the file `thresholding.m` (See [Appendix B](#)) the optimal threshold values were printed to the console:

```

Optimal threshold for img3.png: 177
Optimal threshold for img4.png: 129

```

4. Develop code to convert a grayscale image to a binary image using a given threshold value. Do **NOT** use internal functions for this (e.g., the MATLAB function *im2bw*).

```

% Q4: Function to convert grayscale to binary using threshold
function binary_img = grayscale_to_binary(img, threshold)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [rows, cols] = size(img);

    % Initialize binary image
    binary_img = zeros(rows, cols);

    % Apply threshold manually using loops
    for i = 1:rows
        for j = 1:cols
            if img(i, j) > threshold
                binary_img(i, j) = 255; % White pixel
            else
                binary_img(i, j) = 0; % Black pixel
            end
        end
    end
end
end

```

```

    % Convert to uint8
    binary_img = uint8(binary_img);
end

```

5. Use the optimal thresholds to generate and display binary image for *img3.png* and *img4.png*.

See Figure 11 and Figure 12 below.

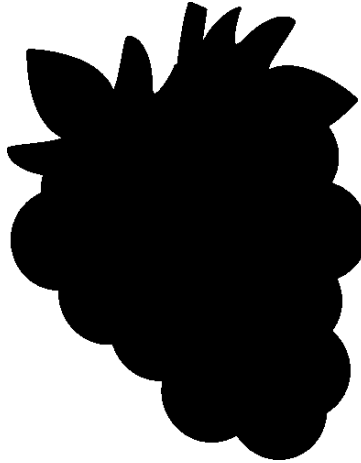


Figure 11: binary image of *img3.png* with *threshold* = 177



Figure 12: binary image of *img4.png* with *threshold* = 129

Appendix A

This section contains the full code for both Spatial Filtering questions ([Smoothing](#) & [Sharpening](#)).

Appendix B

This section contains the full code implemented for the [Thresholding](#) section. It is identical to the submitted file `thresholding.m`.

```

% ECE 7410 - Laboratory 2: Thresholding

```

```

%% Q2: Function to calculate optimal threshold using Otsu's method
function optimal_threshold = calculate_optimal_threshold(img)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

```

```

% Convert to double for calculations
img = double(img);

% Calculate histogram
[counts, ~] = hist(img(:), 0:255);

% Calculate probabilities
p = counts / sum(counts);

% Initialize variables
L = 256; % Number of gray levels (0-255)
max_variance = 0;
optimal_threshold = 0;

% Try all possible thresholds
for t = 1:(L - 1)
    % Calculate class probabilities
    omega_0 = sum(p(1:t));
    omega_1 = sum(p(t + 1:L));

    % Skip if one class is empty
    if omega_0 == 0 || omega_1 == 0
        continue;
    end

    % Calculate class means
    mu_0 = sum((0:(t - 1)) .* p(1:t)) / omega_0;
    mu_1 = sum((t:(L - 1)) .* p(t + 1:L)) / omega_1;

    % Calculate inter-class variance
    sigma_b_squared = omega_0 * omega_1 * (mu_0 - mu_1) ^ 2;

    % Update optimal threshold if this variance is larger
    if sigma_b_squared > max_variance
        max_variance = sigma_b_squared;
        optimal_threshold = t - 1; % Convert to 0-255 range
    end
end
end

%% Q3: Calculate optimal thresholds for test images
% Read images
img3 = imread('./L2/img3.png');
img4 = imread('./L2/img4.png');

% Calculate optimal thresholds
threshold_img3 = calculate_optimal_threshold(img3);
threshold_img4 = calculate_optimal_threshold(img4);

% Display results
fprintf('Optimal threshold for img3.png: %d\n', threshold_img3);
fprintf('Optimal threshold for img4.png: %d\n', threshold_img4);

%% Q4: Function to convert grayscale to binary using threshold

```

```

function binary_img = grayscale_to_binary(img, threshold)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [rows, cols] = size(img);

    % Initialize binary image
    binary_img = zeros(rows, cols);

    % Apply threshold manually using loops
    for i = 1:rows
        for j = 1:cols
            if img(i, j) > threshold
                binary_img(i, j) = 255; % White pixel
            else
                binary_img(i, j) = 0; % Black pixel
            end
        end
    end

    % Convert to uint8
    binary_img = uint8(binary_img);
end

%% Q5: Generate and save binary images using optimal thresholds
% Create output directory
if ~exist('./L2/outputs/03_thresholding', 'dir')
    mkdir('./L2/outputs/03_thresholding');
end

% Convert to binary using optimal thresholds
binary_img3 = grayscale_to_binary(img3, threshold_img3);
binary_img4 = grayscale_to_binary(img4, threshold_img4);

% Display and save results
figure('Position', [100, 100, 1200, 400]);

% Display img3 results
subplot(2, 3, 1);
imshow(img3);
title('Original img3.png');

subplot(2, 3, 2);
if size(img3, 3) == 3
    imshow(rgb2gray(img3));
else
    imshow(img3);
end

```

```
title('Grayscale img3.png');

subplot(2, 3, 3);
imshow(binary_img3);
title(sprintf('Binary img3 (threshold = %d)', threshold_img3));

% Display img4 results
subplot(2, 3, 4);
imshow(img4);
title('Original img4.png');

subplot(2, 3, 5);
if size(img4, 3) == 3
    imshow(rgb2gray(img4));
else
    imshow(img4);
end
title('Grayscale img4.png');

subplot(2, 3, 6);
imshow(binary_img4);
title(sprintf('Binary img4 (threshold = %d)', threshold_img4));

% Save binary images
imwrite(binary_img3, './L2/outputs/03_thresholding/bin3.png');
imwrite(binary_img4, './L2/outputs/03_thresholding/bin4.png');

% Save the figure
saveas(gcf, './L2/outputs/03_thresholding/thresholding_results.png');

fprintf('Binary images saved to ./outputs/03_thresholding/\n');
```